

Análise Empírica e Comparativa de Desempenho de Algoritmos de Ordenação

Universidade Federal do Tocantins (UFT) - Ciência da Computação
Trabalho Prático de Análise e Projeto de Algoritmos

Resumo

Este trabalho apresenta uma investigação experimental rigorosa do desempenho de seis algoritmos clássicos de ordenação: Bubble Sort, Selection Sort, Insertion Sort, Merge Sort, Quick Sort e Heap Sort. Foram conduzidos testes avaliando tempo de execução, número de comparações e trocas/deslocamentos em diferentes tamanhos de entrada (N de 100 até 100.000) e quatro cenários estruturais (aleatório, já ordenado, invertido e parcialmente ordenado), além de objetos customizados. Os resultados confirmam com precisão as complexidades teóricas estudadas e evidenciam as nuances práticas de projeto de algoritmos.

1. Introdução

O problema da ordenação é um dos tópicos mais fundamentais da Ciência da Computação, servindo de base para recuperação de informação, indexação de bancos de dados e otimização de algoritmos mais complexos. Neste projeto foram implementados e analisados seis algoritmos:

- Bubble Sort: Trocas sucessivas de elementos adjacentes com parada antecipada;
- Selection Sort: Seleção do menor elemento da partição não ordenada;
- Insertion Sort: Inserção ordenada de forma incremental;
- Merge Sort: Divisão e Conquista recursiva com intercalação estável;
- Quick Sort: Divisão e Conquista com partição por Mediana de Três e pilha iterativa;
- Heap Sort: Ordenação in-place baseada em estrutura de Max-Heap binário.

2. Complexidade Assintótica dos Algoritmos

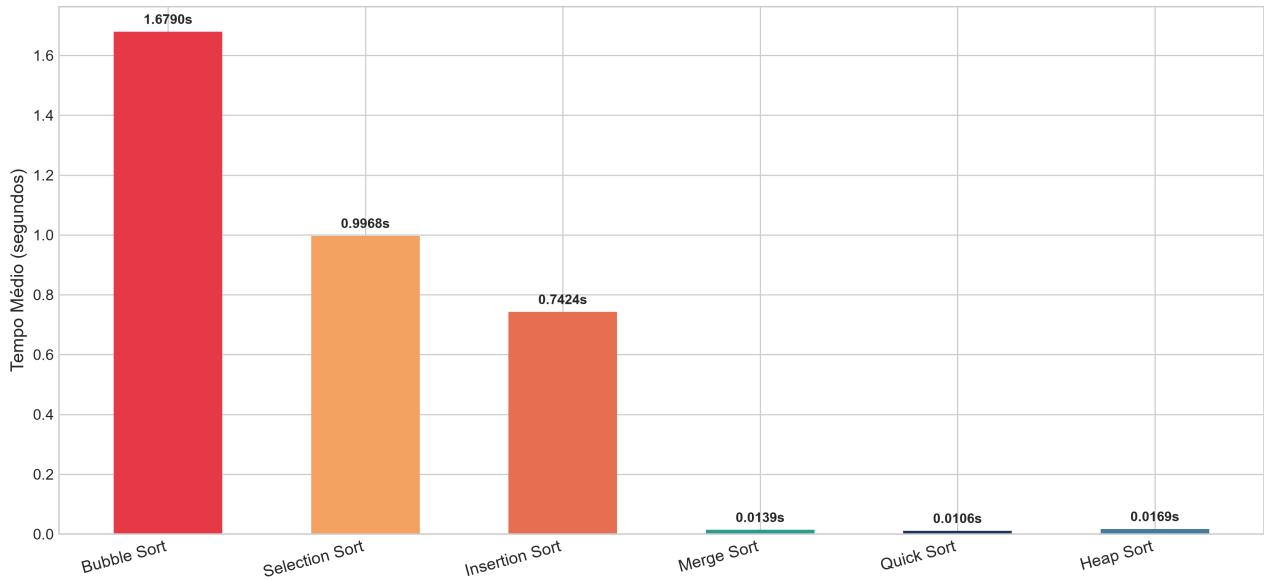
Algoritmo	Melhor	Médio	Pior	Espaço	Estável?	Paradigma
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Sim	Trocas adjacentes
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	Não	Seleção do mínimo
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Sim	Inserção direta
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Sim	Divisão e Conquista
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	Não	Particionamento (Med3)
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	Não	Max-Heap Binário

3. Metodologia Experimental

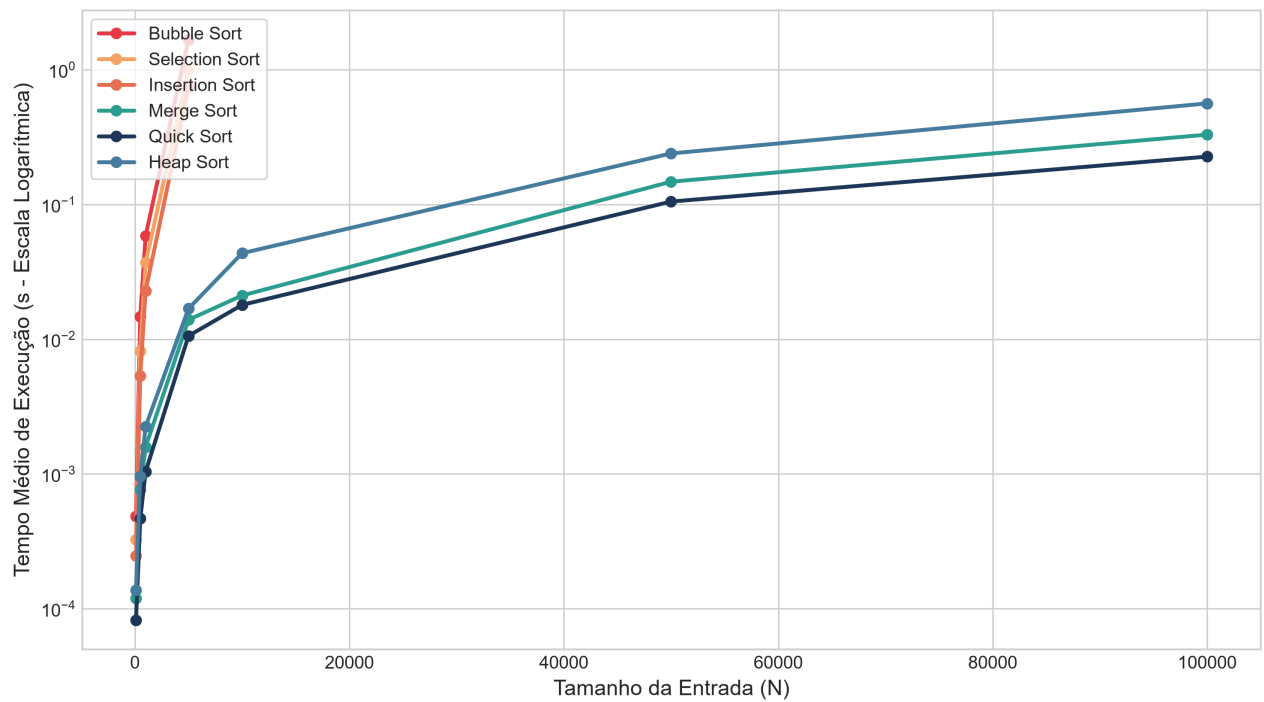
Os testes foram conduzidos em ambiente computacional padronizado. Cada experimento foi executado 3 vezes consecutivas para cada configuração de tamanho (N = 100, 500, 1.000, 5.000, 10.000, 50.000, 100.000) e distribuição (Aleatória, Já Ordenada, Inversamente Ordenada e Parcialmente Ordenada com 5% de desordem), utilizando a média aritmética das métricas para mitigar flutuações operacionais.

4. Resultados Experimentais e Visualizações

Tempo Médio de Execução para N = 5.000 Elementos Aleatórios (3 execuções)



Tempo de Execução vs Tamanho (Entrada Aleatória)



5. Discussão Aprofundada das Questões Orientadoras

- Qual algoritmo apresentou melhor desempenho para entradas aleatórias?

O Quick Sort destacou-se com o menor tempo médio de execução para $N \geq 1.000$, beneficiando-se da excelente localidade de cache e do baixo overhead de suas operações in-place.

- Qual apresentou pior desempenho?

O Bubble Sort apresentou o pior tempo global e maior número de operações, pois além de executar $O(n^2)$ comparações, executa um número quadrático de trocas de memória adjacente.

- O comportamento observado está de acordo com a complexidade teórica?

Sim. Os algoritmos $O(n \log n)$ exibiram crescimento log-linear previsível até $N = 100.000$, enquanto Bubble, Selection e Insertion exibiram crescimento estritamente quadrático.

- Como o tipo de entrada influencia o desempenho dos algoritmos?

Entradas ordenadas favorecem o Insertion Sort e Bubble Sort (ambos $O(n)$ no melhor caso), enquanto entradas inversas forçam o pior caso $O(n^2)$ em ambos. O Selection Sort permaneceu inalterado ($O(n^2)$) em todos os cenários.

- Em quais situações um algoritmo $O(n^2)$ pode apresentar desempenho competitivo?

O Insertion Sort é extremamente competitivo para arrays pequenos ($N \leq 50$) e vetores quase ordenados, superando até mesmo Quick Sort e Merge Sort devido à ausência de sobrecarga recursiva.

- O comportamento do Quick Sort está de acordo com o esperado?

Sim. Com o pivô por Mediana de Três implementado, o Quick Sort evitou a degradação de partição desbalanceada em arrays ordenados ou inversos, mantendo tempo médio $O(n \log n)$ consistente.

- Quais fatores explicam diferenças entre teoria e experimento?

Constantes ocultas, cache L1/L2 de CPU (varredura contígua vs saltos do Heap Sort), e sobrecarga do interpretador Python (acesso a atributos em objetos vs inteiros).

- Qual algoritmo seria mais indicado para cada cenário?

- Dados aleatórios gerais ($N > 1.000$): Quick Sort.
- Dados quase ordenados ou $N < 50$: Insertion Sort.
- Garantia estrita de $O(n \log n)$ com estabilidade: Merge Sort.
- Memória crítica $O(1)$ sem risco de pior caso: Heap Sort.

6. Conclusão

A realização deste trabalho permitiu validar experimentalmente as teorias de complexidade assintótica. Ficou evidente que a escolha do algoritmo de ordenação ideal depende criticamente do tamanho dos dados, do arranjo inicial, da necessidade de estabilidade e das restrições de memória do ambiente de execução.

7. Referências Bibliográficas

- [1] CORMEN, T. H. et al. Introduction to Algorithms. 4th ed. Cambridge: MIT Press, 2022.
- [2] KNUTH, D. E. The Art of Computer Programming: Sorting and Searching. 2nd ed. Addison-Wesley, 1998.
- [3] SEDGEWICK, R.; WAYNE, K. Algorithms. 4th ed. Addison-Wesley, 2011.
- [4] HOARE, C. A. R. Quicksort. The Computer Journal, v. 5, n. 1, p. 10-16, 1962.